PROPUESTA DE DESARROLLO

Square to the Rescue (STTR)

Juego web indie minimalista en JavaScript

Un bullet-hell roguelike donde eres un cuadrado y cada bala toma la forma de quien la dispara. Todo se dibuja y se escucha gracias al código: sin imágenes, sin archivos de audio y sin motor de juego.

Autor	Efrén Campuzano
Materia	[Materia]
Profesor(a)	[Nombre del profesor(a)]
Institución	[Institución]
Grupo	[Grupo]
Fecha	[Fecha de entrega]

Enlaces del proyecto

Jugar en línea (GitHub Pages) https://vidanj.github.io/STTR-Square-to-the-Rescue/

Repositorio en GitHub https://github.com/vidanj/STTR-Square-to-the-Rescue

README en español (README.es.md) https://github.com/vidanj/STTR-Square-to-the-Rescue/blob/main/README.es.md

README en inglés (README.md) https://github.com/vidanj/STTR-Square-to-the-Rescue/blob/main/README.md

Contenido

1. Resumen
2. Concepto del juego
3. Objetivos
4. Público objetivo y plataforma
5. Mecánicas de juego
6. Estética minimalista
7. Audio: sencillo, sintetizado y con efectos
8. Tecnología y arquitectura
9. Metodología y cronograma
10. Publicación y licencia
11. Alcance y trabajo futuro
12. Referencias

1. Resumen

Square to the Rescue (STTR) es un juego web indie del género *bullet-hell roguelike* que se juega directamente en el navegador, sin instalar nada, desde GitHub Pages. El jugador es un cuadrado que debe atravesar un sector de salas generado a partir de una semilla, esquivando los patrones de balas de enemigos que tienen cualquier otra forma geométrica, hasta derrotar a la jefa final.

El juego es **minimalista por diseño**: todo lo que se ve son figuras geométricas dibujadas con código en un canvas de HTML5, y todo lo que se oye —los efectos y la música— se sintetiza en tiempo real con la Web Audio API. No hay imágenes, archivos de sonido ni motor de juego: solo código, publicado completo y con licencia libre.

Este documento presenta la propuesta del proyecto y describe el juego tal como quedó implementado y publicado: su concepto, mecánicas, estilo visual y sonoro, arquitectura técnica, proceso de desarrollo y publicación. La documentación técnica de referencia vive en los README del repositorio, en español e inglés.

Ficha técnica

Género	Bullet-hell roguelike (shoot 'em up vertical con mapa de rutas)
Plataforma	Navegador web de escritorio; teclado (el mouse también dispara y lanza bombas)
Lenguaje	JavaScript ES2022, escrito en TypeScript
Gráficos	HTML5 Canvas 2D; solo figuras geométricas, partículas y sacudida de pantalla
Audio	Web Audio API; efectos y música 100 % sintetizados, sin archivos de sonido

Herramientas	Vite (compilación), Vitest (pruebas), GitHub Actions + GitHub Pages (publicación)
Tamaño publicado	Un solo archivo JavaScript de ≈ 80 KB (≈ 27 KB comprimido)
Código fuente	≈ 6500 líneas en 50 módulos, más 108 pruebas unitarias
Licencia	MIT (código libre)
Jugar	https://vidanj.github.io/STTR-Square-to-the-Rescue/

2. Concepto del juego

2.1 Premisa

Eres **Squero**, un simple cuadrado. Todos los enemigos son otras figuras: círculos, triángulos, diamantes, pentágonos, hexágonos y estrellas. Tu misión es cruzar el sector sala por sala, hacerte más fuerte con mejoras y derrotar a **Astra, la de Cinco Puntas**. En el camino te encuentras con **Bar**, un rectángulo amistoso que atiende la tienda, y con **Prism, el Prisimp**, un mini-jefe que no se deja olvidar.

2.2 La regla central: la forma dice el bando

> **Cada bala toma la forma de quien la disparó.** Tus disparos son cuadraditos; los de un enemigo triangular son triángulos. En una pantalla llena de proyectiles, amigo y enemigo se distinguen de un vistazo, sin depender del color.

La regla se aplica en todo el juego. El jugador y sus balas son **siempre** cuadrados; ningún enemigo puede ser cuadrado (`ENEMY_SHAPES` en `src/config.ts`), y los rectángulos se reservan para personajes amistosos. El audio sigue la misma lógica: cada figura suena con su propia forma de onda (sección 7), así que el bando también se reconoce de oído.

2.3 Pilares de diseño

- **Legibilidad ante todo.** Forma = bando, balas sin resplandor y un hitbox diminuto que se revela al enfocar.
- **Partidas cortas y siempre distintas.** Mapa generado con semilla, rutas a elegir y mejoras que solo duran la partida.
- **Minimalismo real.** Nada de recursos externos: geometría y síntesis de sonido escritas en código.
- **Decisiones con peso.** Cada sala da una mejora; las vidas y bombas se compran con la puntuación.

3. Objetivos

3.1 Objetivo general

Desarrollar y publicar, de forma individual, un juego web indie básico y minimalista en JavaScript, jugable en línea desde GitHub Pages, con audio sencillo de efectos, con su código fuente completo liberado en un repositorio público y acompañado de un video corto que explique el código.

3.2 Objetivos específicos y cómo se cumplen

Requisito	Cómo se cumple en STTR
Juego web en JavaScript	Corre en cualquier navegador moderno. El código TypeScript se compila a JavaScript ES2022 estándar con Vite (ver 8.2).
Indie, básico y minimalista	Proyecto individual, sin motor de juego ni recursos gráficos: solo figuras geométricas dibujadas en un canvas.
Repositorio con README	Repositorio público con README en español e inglés: cómo jugar, controles y arquitectura.
Disponible en GitHub Pages	Cada push a `main` se compila y publica con GitHub Actions. Enlace para jugar al inicio de ambos README.
Código completo y liberado	Todo el código fuente, las pruebas y el flujo de despliegue están en el repositorio bajo licencia MIT.
Audio sencillo con efectos	17 efectos de sonido y música por pantalla, sintetizados con la Web Audio API; tecla M para silenciar y −/+ para el volumen.
Video corto explicativo	Video de ≈ 5 minutos que muestra el juego y recorre el código (guion en `docs/guion-video.md`).

4. Público objetivo y plataforma

- **Jugadores casuales y aficionados** a los shoot 'em up y roguelikes que buscan partidas cortas, rejugables y sin instalación.
- **Plataforma:** navegador de escritorio (Chrome, Edge, Firefox o Safari). No requiere cuenta, descarga ni plugins.
- **Controles:** teclado, con apoyo de mouse para disparar y lanzar bombas.
- **Pantalla:** resolución lógica vertical de 480 × 720 píxeles, como los clásicos del género, que se escala para ocupar la ventana.

5. Mecánicas de juego

5.1 Controles

Acción	Teclas
Moverse	WASD / flechas
Disparar	Z / Espacio / clic izquierdo
Enfoque (lento y muestra el hitbox)	Shift
Bomba (cuadrado que crece)	X / clic derecho
Pausa	Esc

Acción	Teclas
Confirmar / empezar / reintentar	Enter
Elegir mejora	1 / 2 / 3
Mapa: seleccionar y entrar a una sala	← / → (o A / D) y Enter / Z / Espacio
Escena: avanzar diálogo	Z / Enter / Espacio / clic izquierdo
Silenciar / activar sonido	M
Bajar / subir volumen	– / + (también en el teclado numérico)

5.2 Estructura de una partida

1. **Mapa del sector.** Cada partida empieza con un mapa de salas por capas, generado a partir de una semilla: fácil → media → media → difícil → difícil → jefe, con dos o tres salas por capa. El jugador elige su ruta entre salas conectadas.
2. **Salas.** Cada sala es una fase de bullet-hell con oleadas de enemigos. Superarla otorga **una mejora** a elegir entre tres cartas.
3. **Mini-jefe.** En la tercera capa espera **Prism, el Prisimp** (sala élite, **dos** mejoras). Todas las rutas lo ofrecen; si lo evitas, te sigue y aparece justo antes del jefe final.
4. **The Bar.** Una vez por partida, una sala de las capas difíciles es la tienda de **Bar**, donde se gasta la puntuación en vidas y bombas en lugar de combatir.
5. **Jefa final. Astra, la de Cinco Puntas**, pelea en tres fases (*Overture*, *Spin Cycle* y *Supernova*) y pide refuerzos en cada cambio. Los jefes hacen una entrada: llegan, muestran su barra de vida y son inmunes un momento antes de atacar.

5.3 Contenido

Elemento	Cantidad	Detalle
Salas diseñadas	10	3 fáciles, 3 medias, 2 difíciles, 1 élite (Prism) y 1 de jefe (Astra)
Figuras enemigas	6	Círculo, triángulo, diamante, pentágono, hexágono y estrella
Patrones de disparo	5	Apuntado, anillo, espiral, abanico (arc) y ráfaga (burst)
Mejoras	8	2 de estadísticas y 6 tipos de disparo con niveles
Artículos de tienda	2	Repair Plating (+1 vida) y Bomb Cache (+1 bomba)
Personajes con escenas	4	Squero, Bar, Prism y Astra

5.4 Supervivencia: hitbox, enfoque y bomba

- El **hitbox real** es solo el punto del centro del cuadrado. Mantener **Shift** reduce la velocidad y lo hace visible para esquivar patrones densos. Rozar las balas es seguro.

- Se empieza con **3 vidas y 3 bombas**. Recibir un golpe cuesta una vida y da **3 segundos de invencibilidad** parpadeante; las balas no desaparecen, así que hay que aprovechar ese margen para escapar.
- La **bomba** es un cuadrado que se lanza hacia arriba y **crece exponencialmente** mientras avanza, borrando las balas enemigas que abarca y dañando a los enemigos que envuelve. Lanzada desde muy abajo llega casi al ancho de la pantalla.

5.5 Mejoras y flujos de disparo

Las mejoras duran **solo la partida actual** (roguelike). Todas disparan cuadrados; lo que cambia es cómo se comportan:

Mejora	Efecto
Rapid Fire	Dispara 18 % más rápido.
Sharpened Edge	+1 de daño a todos los flujos.
Piercing Rounds	Un flujo que atraviesa enemigos; en niveles altos gana daño con cada uno.
Ricochet	Disparos que rebotan en paredes y en enemigos, golpeando a cada uno.
Rotating Shot	El disparo principal gira (daño ×1.5) y después crece mientras vuela.
Scatter	Cono de perdigones a corta distancia que luego rebotan en las paredes.
Homing Splinter	Astillas lentas que persiguen al enemigo más cercano.
Orbit Shield	Un cuadrado que orbita al jugador y se come las balas enemigas.

Los disparos **primero se profundizan y luego se amplían**: elegir otra vez un disparo que ya tienes lo sube de nivel (hasta 4; el escudo orbital hasta 6) y cada nivel cambia su comportamiento. Solo cuando ese flujo está al máximo, una nueva elección añade otro flujo de nivel 1. Ninguna elección se desperdicia.

5.6 La puntuación es la moneda

Las vidas y las bombas nunca son recompensas de sala: se compran en The Bar con la puntuación, lo que obliga a decidir entre arriesgar por más puntos o asegurar la supervivencia.

Artículo	Efecto	Precio	Máximo
Repair Plating	+1 vida	2500 (+900 por cada compra repetida)	5 vidas
Bomb Cache	+1 bomba	1200 (+400 por cada compra repetida)	6 bombas

5.7 Historia

Breves **escenas estilo novela visual** presentan al elenco en los primeros encuentros, las entradas y las derrotas de los jefes. Cada escena se reproduce una sola vez por partida y está escrita como datos en `src/content/cutscenes.ts`.

6. Estética minimalista

El estilo visual sale de una restricción: **no usar ningún recurso gráfico**. Cada personaje, bala, partícula y pantalla se dibuja con trazos de canvas desde `src/render/Renderer.ts`, que centraliza el dibujo de figuras, el resplandor, el escalado a la ventana y la sacudida de pantalla.

- **Geometría pura.** Ocho figuras (`Shape` en `src/config.ts`) bastan para todo el elenco.
- **Paleta neón sobre fondo casi negro**, con una cuadrícula tenue de fondo.
- **Balas sin resplandor.** Los cuerpos brillan, pero los proyectiles son rellenos nítidos: con la pantalla llena, el brillo destruiría la legibilidad.
- **Una sola superficie.** HUD, mapa, tienda y escenas se dibujan en el mismo canvas; el HTML solo contiene el canvas y una pantalla de carga hecha con CSS (un cuadrado que gira).

Elemento	Color	Muestra
Fondo	#05060d	
Jugador (Squero)	#6cf0ff	
Balas del jugador	#aef7ff	
Enemigos	#ff5bd0	
Balas enemigas	#ffcf5b	
Jefes	#ff6b6b	
Acento de interfaz	#8f7bff	

7. Audio: sencillo, sintetizado y con efectos

El audio se resuelve sin un solo archivo de sonido: cada efecto y cada nota de la música se generan en tiempo real con la **Web Audio API** del navegador (`src/audio/`). Así el juego sigue pesando unos pocos kilobytes y el sonido respeta la misma regla visual del juego.

7.1 Cada figura suena con su forma de onda

El mapa `SHAPE_TIMBRE` (`src/audio/timbres.ts`) asigna a cada figura un timbre. Los disparos, impactos y la muerte de un enemigo comparten su voz, así que el bando se distingue también de oído.

Figura	Forma de onda	Carácter
Cuadrado (jugador)	Cuadrada	Exclusiva del jugador: ninguna figura enemiga la comparte
Círculo	Senoidal	Pura y suave
Triángulo	Triangular	Zumbido ligero
Diamante	Diente de sierra	Brillante y áspera
Pentágono	Personalizada	Refuerza el 5.º armónico: hueca, algo nasal

Figura	Forma de onda	Carácter
Hexágono	Personalizada	Refuerza el 6.° armónico: zumbido de lengüeta
Estrella (jefa)	Personalizada	Todos los armónicos, con picos en el 5.° y 10.°: filo áspero
Rectángulo (Bar)	Casi senoidal	Amistosa: apenas un susurro de armónicos

7.2 Dos primitivas para todo

Todos los sonidos se construyen con dos funciones de `src/audio/synth.ts`:

- `tone()` — una nota de oscilador con envolvente de ataque-sostén-caída, filtro opcional, deslizamiento de tono y vibrato.
- `noise()` — un golpe de ruido blanco filtrado, para explosiones, impactos y percusión.

Cada voz programa su envolvente en el reloj exacto del `AudioContext` y se desconecta sola al terminar, de modo que el grafo de audio nunca acumula nodos muertos.

7.3 Catálogo de efectos

El código del juego solo **nombra** el efecto en el lugar donde ocurre el evento, por ejemplo `world.audio.sfx.play("kill", { shape })`; la receta de cómo suena vive en `src/audio/sfx.ts`. Hay 17 efectos:

Tipo	Efectos
Combate (10)	disparo, disparo enemigo, impacto, baja, bala absorbida, daño recibido, bomba, llegada del jefe, cambio de fase, derrota del jefe
Interfaz (7)	mover selección, confirmar, elegir mejora, comprar, compra denegada, avanzar diálogo, pausa

Para que la densidad del bullet-hell no se convierta en un muro de ruido, un **limitador** (`Throttle`) impone un tiempo mínimo entre repeticiones del mismo efecto (por ejemplo, 50 ms para el disparo), hay un máximo de **24 voces** simultáneas (los efectos menores se descartan primero) y cada reproducción varía su tono ±3 % para no sonar mecánica.

7.4 Música por pantalla

La banda sonora son cuatro bucles y tres ráfagas, todos en la familia de La menor para que los fundidos encajen. Las pistas se escriben como **patrones de texto**, un símbolo por semicorchea (`src/audio/tracks.ts`):

```
{ inst: "pad", notes: "a3*16 a3*16 g3*16 g3*16 a3*16 a3*16 a3*16 g#3*16" },
{ inst: "hat", notes: rep(".*2 +*2", 4), gain: 0.6, minIntensity: 2 },
{ inst: "kick", notes: "x*8 x*8", gain: 0.35, minIntensity: 2 },
```

Pista	Dónde suena	Comportamiento
Grid Hymn	Menú, mapa, mejoras y victoria	Tema tranquilo de cuadrícula

Pista	Dónde suena	Comportamiento
Patrol	Salas de combate	Se intensifica con la dificultad de la sala
Nexus	Jefes	Escala en cada fase del jefe
Bar	La tienda	Bucle de lounge
Ráfagas	Sala superada, derrota, victoria	Se reproducen una vez sobre la música

`Game.syncMusic` elige la pista según el estado del juego y un secuenciador con anticipación (`MusicPlayer`) hace fundidos cruzados de 0.8 s entre bucles.

7.5 Control del jugador y reglas del navegador

- **M** silencia; **−** y **+** ajustan el volumen. La preferencia se recuerda entre visitas.
- Los navegadores bloquean el audio hasta que el usuario interactúa, así que el sonido se desbloquea con la primera tecla o clic.
- El audio **nunca** usa el generador aleatorio de la partida: jugar con o sin sonido produce exactamente la misma partida.

8. Tecnología y arquitectura

8.1 Stack

Capa	Tecnología	Uso
Lenguaje	TypeScript 5 → JavaScript ES2022	Todo el código del juego
Gráficos	Canvas 2D API	Dibujo de figuras, partículas, HUD y pantallas
Audio	Web Audio API	Síntesis de efectos y música
Compilación	Vite 5	Servidor de desarrollo y paquete de producción
Pruebas	Vitest 2	108 pruebas unitarias
Publicación	GitHub Actions + GitHub Pages	Compilar y publicar en cada push a main

El juego no tiene **ninguna dependencia en tiempo de ejecución**: las tres dependencias del proyecto (TypeScript, Vite y Vitest) son solo de desarrollo.

8.2 JavaScript y TypeScript

> La propuesta pide un juego en JavaScript. STTR está escrito en **TypeScript**, que es JavaScript con anotaciones de tipos. El navegador nunca ejecuta TypeScript: Vite elimina los tipos y genera un único archivo **JavaScript estándar (ES2022)**, que es lo que se publica en GitHub Pages.

Los tipos se eligieron para mantener ordenado un proyecto de unas 6500 líneas: errores como usar una figura que no existe, olvidar un estado del juego o pasar parámetros incorrectos a un patrón de disparo se detectan al compilar (`npm run typecheck`) y no a mitad de una partida.

8.3 Estructura del código

Carpeta	Responsabilidad
`src/core/`	Bucle de paso fijo (`Loop`), entrada (`Input`), generador con semilla (`RNG`), pool de objetos (`Pool`), máquina de estados (`Game`), estado de la partida (`RunState`) y la sala activa (`World`).
`src/map/`	Grafo de salas por capas (`GameMap`) y su generación con semilla (`MapGen`).
`src/entities/`	`Player`, `Enemy`, `Boss`, `BombSquare` y el modelo de datos `Bullet`.
`src/systems/`	Balas con pool, colisiones, partículas, oleadas, mejoras y tienda.
`src/content/`	Contenido como datos: patrones, mejoras, escaleras de disparo, tienda, sectores, escenas y salas (`stages/`).
`src/ui/`	HUD, superposiciones, mapa, tienda y reproductor de escenas.
`src/render/`	Canvas, escalado, sacudida de pantalla y dibujo de figuras.
`src/audio/`	Motor de audio, efectos, música, instrumentos, timbres y las primitivas de síntesis.

8.4 Decisiones técnicas clave

Bucle de paso fijo a 60 Hz (`src/core/Loop.ts`). El tiempo real se acumula y la lógica avanza en pasos exactos de 1/60 s; el dibujo ocurre una vez por cuadro. Así los patrones de balas son deterministas y la dificultad no depende de los hercios del monitor:

```
this.accumulator += frameTime;
while (this.accumulator >= DT && steps < MAX_STEPS) {
  this.step();              // lógica: siempre 1/60 s
  this.accumulator -= DT;
  steps++;
}
this.render(this.accumulator / DT);   // dibujo: una vez por cuadro
```

- **Máquina de estados** (`src/core/Game.ts`): menú, mapa, escena, sala, tienda, mejora, pausa, derrota y victoria. El combate solo avanza en el estado de juego.
- **Aleatoriedad con semilla** (`src/core/RNG.ts`, algoritmo mulberry32). Toda la aleatoriedad que afecta la partida pasa por aquí, nunca por `Math.random`; una semilla reproduce exactamente el mismo mapa y las mismas cartas.
- **Pools de objetos** (`src/core/Pool.ts`). Balas y partículas se reciclan en lugar de crearse, y se retiran intercambiándolas con la última, sin generar basura para el recolector aunque haya cientos en pantalla.
- **Colisiones círculo contra círculo** (`circleHit` en `src/systems/CollisionSystem.ts`). Todas las figuras se tratan como círculos de su radio, la sensación estándar y justa del género.

- **Contenido como datos.** Salas, patrones, mejoras, tienda, sectores y escenas son estructuras en `src/content/`; ajustar la dificultad o añadir una sala no requiere tocar el motor.

8.5 Calidad

El proyecto se compila con TypeScript en modo estricto y tiene **108 pruebas unitarias en 13 archivos** (Vitest). Cubren, entre otras cosas, que el mapa sea reproducible a partir de su semilla, las escaleras de disparo, los precios de la tienda, las escenas, los jefes y el audio, este último con un `AudioContext` simulado para probar sin navegador.

9. Metodología y cronograma

El juego se construyó de forma **iterativa e incremental**: primero un núcleo jugable completo y después mejoras pequeñas, cada una probada (pruebas unitarias y partida manual) y publicada automáticamente en GitHub Pages. El historial de commits del repositorio documenta las fases:

Fecha	Fase	Resultado
27 jul 2026	Versión inicial (v01)	Núcleo jugable: bucle de paso fijo, jugador, enemigos y jefes, balas, colisiones, mapa con semilla, mejoras y primeras escenas.
27–28 jul 2026	Despliegue	Flujo de GitHub Actions y primera publicación en GitHub Pages.
28 jul 2026	Pantalla de carga	Indicador animado mientras arranca el juego.
30 jul 2026	Rebalanceo y contenido	Escaleras de nivel para los disparos, nuevas mejoras (Scatter, Homing Splinter, Orbit Shield), tienda The Bar, sectores, entradas de jefe y escenas pulidas.
12 nov 2026	Audio y documentación	Efectos y música sintetizados, silencio y volumen; README en español.
12 nov 2026	Entrega	Licencia MIT, esta propuesta y el guion del video explicativo.

10. Publicación y licencia

10.1 Repositorio y despliegue

El código completo está en un repositorio público de GitHub. El archivo `.github/workflows/deploy.yml` publica el juego automáticamente:

1. Se activa con cada push a la rama `main` (o manualmente).
2. Instala Node.js y las dependencias con `npm ci`.
3. Ejecuta `npm run build`, que verifica los tipos y genera la carpeta `dist/`.
4. Sube `dist/` como artefacto y lo despliega en GitHub Pages.

Vite está configurado con rutas relativas (`base: "./"`), por lo que el juego funciona dentro de la subruta del proyecto en GitHub Pages. Una pantalla de carga cubre el arranque.

10.2 Ejecutarlo localmente

```
npm install
npm run dev        # abre http://localhost:5173
npm test           # 108 pruebas con Vitest
npm run build      # verificación de tipos + paquete de producción
```

10.3 Licencia

El código se libera bajo la **licencia MIT** (archivo LICENSE): cualquiera puede jugarlo, estudiarlo, modificarlo y redistribuirlo, siempre que conserve el aviso de autoría.

11. Alcance y trabajo futuro

El alcance propuesto (un sector completo con mini-jefe, tienda y jefa final, audio y publicación en línea) está terminado. La arquitectura deja preparado el crecimiento:

- **Más sectores.** La forma del mapa ya se define por sector en `src/content/sectors.ts`; un sector nuevo es otra entrada de datos.
- **Controles táctiles y de gamepad**, para jugar en móviles y tabletas.
- **Tabla de puntuaciones** y partidas compartibles por semilla.
- **Opciones de accesibilidad**, como reducir la sacudida de pantalla o ajustar la velocidad del juego.

12. Referencias

- README del proyecto en español: README.es.md
- README del proyecto en inglés: README.md
- Guion del video explicativo: docs/guion-video.md
- Repositorio: https://github.com/vidanj/STTR-Square-to-the-Rescue
- Juego en línea: https://vidanj.github.io/STTR-Square-to-the-Rescue/
- MDN Web Docs — Web Audio API
- MDN Web Docs — Canvas API
- Referentes de género: los bullet-hell verticales (como la serie *Touhou Project*) y los roguelikes de mapa de rutas (como *Slay the Spire*).